

Tahsin Shuborna

NLP_Driven_Agentic_AI_Systems_for_Software_Engineering...

 CSE4261 Quiz-4 (Assignment) Paper Checking

Document Details

Submission ID

trn:oid:::3618:133164970

Submission Date

Mar 27, 2026, 10:55 AM GMT+6

Download Date

Mar 27, 2026, 10:57 AM GMT+6

File Name

NLP_Driven_Agentic_AI_Systems_for_Software_Engineering_A_Survey_in_Resource_Constrained_....pdf

File Size

272.4 KB

8 Pages

3,920 Words

24,138 Characters

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **1 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 1%  Submitted works (Student Papers)

Match Groups

- 
1 Not Cited or Quoted 1%
 Matches with neither in-text citation nor quotation marks
- 
0 Missing Quotations 0%
 Matches that are still very similar to source material
- 
0 Missing Citation 0%
 Matches that have quotation marks, but no in-text citation
- 
0 Cited and Quoted 0%
 Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 1%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1

Student papers

Northern Technical University on 2026-03-15

1%

NLP Driven Agentic AI Systems for Software Engineering: A Survey in Resource Constrained Development Environments in Bangladesh

Abstract

Imagine a world where intelligent agents can read one's mind, well, almost. Agentic AI is making that world real by thinking, planning and acting on its own, helping developers navigate the maze of software creation. Natural Language Processing (NLP) is the bridge that lets these agents understand instructions, explore documentation and assist in coding, debugging and more. This paper travels through the world of NLP based agentic AI systems, looking at how they work, the data they use and the tasks they tackle. But these systems often need constant internet, heavy computing power and struggle to adapt to changing work environments. Unlike other studies that only show blueprints, the study focuses on real world challenges, especially for freelancers and small companies in places like Bangladesh, where resources are limited. It also explores the path forward, creating lightweight, adaptable and internet smart Agentic AI tools that can actually thrive in the messy, unpredictable world of software development. By connecting AI's potential to practical realities, this work shows how intelligent agents can truly become partners for developers, not just powerful ideas on paper.

Keywords - Agentic AI, Natural Language Processing, Software Engineering, Resource Constrained Environments, Freelance Development, SMEs, Intelligent Software Agents

1 Introduction

Artificial Intelligence (AI) is entering a new era in which machines are no longer limited to executing predefined instructions but are increasingly capable

of understanding goals, making decisions and performing complex tasks. AI is not just a quiet tool sitting behind the screen, waiting for commands. It is now transforming into active agents that can think about problems, plan their next moves and carry out complex tasks with minimal human supervision. This transformation has given rise to the concept of Agentic AI, where AI systems function not merely as text generators but as intelligent agents capable of planning tasks and interacting with tools (?).

In this paradigm, Agentic AI is about building systems that can act on their own. This usually involves multi agent systems (MAS), where specialized agents work together, communicate and coordinate to solve problems too big for any single agent (?). For example, an agentic system designed for automated bug fixing may include a bug analysis agent to examine error logs and identify root causes, a code navigation agent to locate the affected parts of the codebase, a patch generation agent to produce potential fixes, a testing agent to validate the correctness of the solution and a review agent to refine and improve the output. This coordinated workflow demonstrates how agentic systems can transform simple bug reports into reliable, executable software fixes.

To establish a clear conceptual foundation, it is important to distinguish between AI agents and Agentic AI systems. An AI agent refers to a computational entity that can perceive its environment, reason about possible actions and perform tasks in order to achieve a specific objective. In many recent systems, these agents are built on top of Large Language Models that provide reasoning, planning

and communication capabilities (?). Agentic AI, on the other hand, represents a broader architectural paradigm in which one or multiple agents collaborate to accomplish complex goals within a structured workflow (?).

Software engineering is fundamentally driven by language based interaction. Developers communicate requirements, describe bugs, review code, interpret documentation and coordinate tasks primarily through natural language. As a result, the integration of Natural Language Processing (NLP) (?) with agentic architectures has become a key enabler for intelligent software development systems. This language centric nature of software development makes Natural Language Processing (NLP) a critical interface for modern agentic systems. By interpreting requirements, generating code, summarizing documentation and coordinating tasks through language, NLP driven agents can function as collaborative assistants throughout the software development lifecycle.

Table 1 summarizes representative research works on agentic AI systems across different domains. As shown in the table:

- **Scattered Research:** Existing studies are spread across multiple domains, making it difficult to form a unified understanding of agentic system design and evaluation.
- **Limited Focus on Software Engineering:** Most research focuses on domain specific applications or architectures, with fewer studies analyzing support for coding, debugging, testing and deployment.
- **Persistent System Challenges:** Current systems still face issues such as high computational requirements, unreliable outputs and limited scalability.
- **Terminology Ambiguity:** Terms such as AI agents, LLM (?) based agents and multi agent systems are often used interchangeably, creating conceptual confusion (?).
- **Failure Awareness:** Limited attention is given to detecting and recovering from hallucinations, incorrect code generation or tool misuse.

Beyond these research gaps, current surveys often overlook several practical deployment constraints that significantly influence the usability of agentic systems in real world environments.

- **Unstable Internet Connectivity:** Many systems assume continuous internet access, while real world connectivity can be slow or unstable.
- **Freelance Development Workflows:** Freelancers often handle vague client instructions, multiple projects and communication mainly through natural language.
- **Resource Limitations in Small Companies:** SMEs operate with limited computational resources, making large scale agentic systems difficult to adopt.

These challenges are particularly significant in developing countries such as Bangladesh, where internet connectivity varies across regions, freelance software development plays a major role in the digital economy and many companies operate as small or medium sized enterprises. Consequently, designing **resource aware and connectivity resilient agentic systems** is essential for enabling practical deployment in such environments.

To address these limitations, this paper focuses on the role of NLP driven agentic AI systems in software engineering. The main contributions of this paper are summarized as follows:

- Provide a structured review of recent agentic AI systems with a particular focus on their application in software engineering tasks.
- Compare existing agentic systems across different domains and analyze their applicability to software engineering environments.
- Present a simplified taxonomy of agentic architectures based on their functional roles within the software development lifecycle (?).
- Introduce practical design considerations for real world deployment, including connectivity aware agent systems, support for freelance development workflows and lightweight architectures suitable for small and medium sized companies.
- Introduce the concept of **resource aware and connectivity resilient agentic systems** for software engineering environments in developing regions.
- Provide a context aware perspective by discussing how agentic AI systems can be

Table 1: Representative Research on Agentic AI Systems Across Domains

References	Focus	Key Contributions	Limitations
?	Knowledge Graph	Introduces an agentic framework enabling LLMs to interact with graph structured knowledge.	Limited evaluation on large scale knowledge graphs.
?	Healthcare	Proposes an adaptive agent-based system for symptom clarification and disease prediction.	Small dataset size and reliance on synthetic data.
?	Agriculture	Develops a multi-agent framework for autonomous decision support in smart agriculture.	Performance depends on input data reliability.
?	Education	Automates short answer grading using agentic reasoning and RAG.	Difficulty handling highly subjective responses.
?	Education / Syllabus	Multi-agentic AI for automatic course syllabus generation using LangGraph.	Requires high data consistency for accuracy.
?	Cybersecurity	Security framework for home AI to note cyber threats in preventive healthcare.	Vulnerable to specific prompt injection attacks.
?	Student Remediation	Agentic AI for student diagnosis and individualized remediation.	Scalability in diverse educational environments.
?	Gaming / Games	A novel hierarchical agent framework for imperfect-information games.	Complexity in multi-agent strategy coordination.

adapted for developing regions such as Bangladesh, where resource constraints and freelance based development ecosystems are common.

The remainder of this paper is organized as follows. Section 2 provides the background and foundational concepts of Agentic AI and LLM based agents. Section 3 presents the proposed framework for context aware agent collaboration in software engineering. Section 4 provides a comparative analysis of agentic systems focusing on software related applications. Section 5 discusses key challenges in real world environments. Finally, Section 6 concludes the paper and outlines future research directions.

2 Background and Foundations of Agentic AI

2.1 Evolution of Agentic AI

The development of agentic AI can be seen as a natural continuation of the broader evolution of artificial intelligence. Imagine AI as a student in school:

- Early AI (rule-based systems) was like a stu-

dent who could only follow exact instructions from the teacher. If asked “2 + 3,” it answers correctly, but any variation confuses it.

- Machine learning AI became like a student who learns from examples. Even if a new problem appears, it can reason and guess the correct answer.
- Agentic AI is like a young scientist. It does not wait for instructions, it can set its own goals, plan steps, use tools and adapt to new challenges. For example, it could figure out how to build a small robot by reading manuals, testing ideas and improving its design iteratively.

In short, Agentic AI extends traditional AI by combining learning, reasoning and autonomous action, enabling systems to perform complex tasks with minimal human guidance which has been illustrated in Figure 1 (???)

The emergence of Large Language Models (LLMs), particularly transformer based architectures, has significantly accelerated this transition (??). These models provide strong reasoning and language understanding capabilities, which, when

combined with planning mechanisms, memory and external tools, enable the development of autonomous AI agents.

Recent research highlights a shift from static generative models to dynamic agentic architectures (??). For example, agentic frameworks have been proposed for complex engineering tasks such as hardware code generation, where multiple agents collaborate to design and validate system components (?).



Figure 1: Evolution of Artificial Intelligence Toward Agentic Systems

In essence, agentic AI combines the learning ability of modern ML systems with planning, reasoning and tool usage, creating systems that are not only intelligent but also adaptive and capable of long term goal directed behavior. This evolution illustrated in Figure 1 marks a significant step toward autonomous artificial intelligence that can act effectively in real world environments.

2.2 Applications of Agentic AI

Table 2 summarizes key application areas and representative works of Agentic AI systems.

2.3 Taxonomy of Agentic AI Systems

A typical Agentic AI architecture includes several core components such as a planning module, memory system, reasoning engine and action execution layer. Based on existing literature and architectural design patterns, agentic AI systems can be broadly categorized into structural paradigms. Understanding these patterns is important for analyzing how agentic systems can be adapted to practical environments such as software engineering workflows. Figure 2 presents a high level taxonomy of agen-

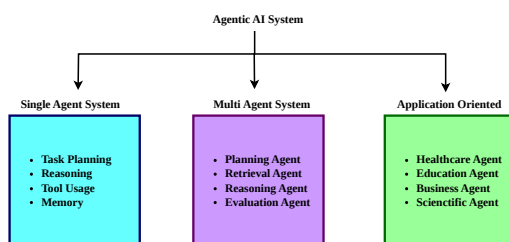


Figure 2: Taxonomy of Agentic AI Architectures

tic AI systems that organizes the current research landscape into three major categories:

Single Agent Systems: A single autonomous agent handles task planning, reasoning and execution, and is effective for moderately complex tasks such as document analysis, code generation and information retrieval.

Multi Agent Systems: Multiple specialized agents collaborate, each responsible for a specific role, enabling the system to solve more complex and long term problems.

Application Oriented Agentic Systems: These systems apply agentic architectures to specific domains such as healthcare, education, optimization and software engineering, with this survey focusing on their role in software engineering workflows.

3 Agentic AI for Software Engineering: An NLP Centric and Resource Aware Perspective

Unlike many existing surveys that primarily focus on architectural designs of agentic systems, this paper examines agentic AI from an NLP driven software engineering perspective with particular attention to resource constrained development environments as depicted in Figure 3. In the world of software engineering, language is code, and understanding it intelligently is where NLP powered agentic AI becomes a game changer. Software engineering workflows are inherently language intensive, involving requirement descriptions, bug reports, documentation, code comments and developer discussions. As a result, Natural Language Processing becomes a critical interface through which agentic systems can understand, coordinate and assist development tasks.

Yet most agentic AI research assumes perfect conditions: stable internet, powerful machines and uninterrupted workflows. In practice, conditions are far from ideal. In countries like Bangladesh, freelance developers often contend with unstable connectivity, SMEs function with limited infrastructure and resources are frequently constrained. In such settings, AI must be smart, lightweight, adaptive. In the following, three deployment perspectives of NLP driven agentic systems have been explored in software engineering environments in Bangladesh.

3.1 Connectivity Aware NLP Agents in Bangladesh

In many parts of Bangladesh, interacting with NLP driven intelligent systems is not always seamless.

Table 2: Application Domains of Agentic AI Systems

Domain	Application Task	Representative Papers
Healthcare	Symptom clarification, clinical decision support, medical report summarization, medical knowledge retrieval, severity detection	(????????)
Education	Short answer grading, personalized tutoring, student performance prediction	(???)
Agriculture	Crop monitoring and plant disease detection	(??)
Software Engineering / Automation	Code generation, bug detection, programming assistance, software automation	(???)
Finance and Business Analytics	Consumer segmentation, banking analytics, strategic planning, risk prediction	(????)
Conversational and LLM Systems	Chatbots, dialogue systems, prompt optimization, LLM orchestration	(?????)
AI Research and Knowledge Systems	Literature review automation, document analysis, research assistant systems, conceptual AI studies	(???)
Optimization and Multi Agent Systems	Multi agent coordination and decision optimization	(??)
Smart Systems (Cities, Robotics, Emergency)	Traffic optimization, autonomous planning, disaster response coordination	(???)
Hardware / Scientific Applications	Circuit design automation, graph learning, brain data analysis, multimodal reasoning	(????)

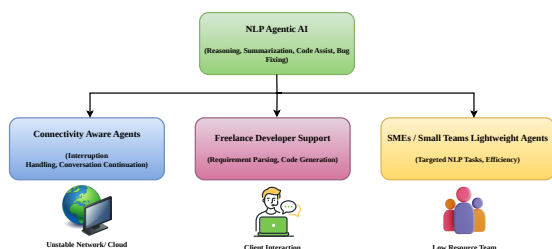


Figure 3: Resource Aware and Connectivity Resilient Agentic Systems

Modern software engineering workflows increasingly rely on natural language interactions, where developers describe requirements, discuss bugs and coordinate development tasks through text. As a result, NLP based agentic AI systems act as conversational interfaces that translate human instructions into executable software actions.

However, in real world environments such interactions are not always stable. Conversations with NLP based agentic systems may be interrupted by unstable networks, delayed responses or sudden disconnections. While the interaction pauses, the development workflow itself continues, creating challenges for language driven agents that rely on continuous dialogue.

Most existing agentic architectures assume persistent connectivity and real time access to large

language models and external tools (??). Recent agentic NLP frameworks such as GraphAgent and RUBRA demonstrate how language models can reason over structured knowledge and retrieved context to support complex tasks (??). However, these systems are typically designed for stable cloud environments, where uninterrupted communication between users and agents is assumed.

This creates a critical mismatch between system design and practical use. In connectivity limited environments, NLP agents should not only understand language but also continue conversations even if the connection is interrupted. These systems should be able to save the conversation, handle incomplete instructions and continue working when the internet is back.

In this situation, NLP is not just for understanding language. It also helps different AI agents work together smoothly, even during connection problems. This makes agentic AI more practical and useful, especially in countries like Bangladesh, where internet and system limitations are common.

3.2 NLP Agentic Support for Freelance Developers

For freelance developers, software engineering is not just about writing code, it is a continuous language driven interaction with clients. Most freelance projects begin with natural language, including client requirements, vague instructions, iterative feedback and evolving expectations. In this dynamic environment, understanding and acting on language becomes as important as technical execution.

NLP based agentic AI systems have the potential to transform this workflow by acting as intelligent intermediaries between human intent and technical implementation. Using large language models and reasoning frameworks, these agents can interpret client instructions, summarize requirements, translate them into structured development plans and generate corresponding code artifacts (??). In addition, recent agentic systems show that language based reasoning and tool use can help with complex development tasks such as debugging, writing documentation and improving code step by step (?).

Despite this strong alignment, most existing studies focus on organizational software pipelines, where tasks are structured and collaboration occurs within teams. Freelance development ecosystems remain largely underexplored, even though they

rely heavily on natural language communication between developers and clients.

From an NLP perspective, freelance workflows present a unique opportunity for agentic systems to operate as collaborative partners rather than simple automation tools. By maintaining conversational context, interpreting evolving requirements and assisting in planning and execution, NLP based agents can support developers throughout the entire project lifecycle. This highlights a practical application of agentic AI in real world development environments, particularly relevant to the growing freelance software economy in developing regions such as Bangladesh.

3.3 Lightweight NLP Agentic Systems for Bangladeshi SMEs

Small and medium sized enterprises (SMEs) are businesses that operate with limited workforce, budget and technical infrastructure. In Bangladesh, a large portion of software development organizations fall into this category, including small startup teams building web applications, local IT service providers and software vendors developing custom solutions for clients. For instance, a small development team working on an E-commerce platform may consist of only a few developers responsible for requirement analysis, coding, testing and client communication simultaneously.

In such environments, software engineering workflows are often fast paced and resource constrained. Developers must handle multiple responsibilities with minimal tooling support. Importantly, natural language interactions play a central role in these workflows, including requirement discussions, client instructions, bug reports and project documentation. From an NLP perspective, these language artifacts form the primary interface through which developers communicate, reason about problems and coordinate development tasks.

However, most existing NLP driven agentic AI systems are designed for large scale infrastructures that assume continuous cloud connectivity and substantial computational resources (??). Such assumptions limit the practical applicability of these systems in SME environments, where computing infrastructure, GPU resources and stable connectivity may be limited.

Rather than maximizing model scale, SME oriented agentic systems require a different design philosophy focused on efficiency and task specialization. Lightweight NLP based agents can be de-

veloped to perform targeted high impact functions such as requirement summarization, documentation assistance, automated bug explanation and code snippet generation. These capabilities can significantly reduce developer workload without requiring large scale infrastructure.

In this case, the goal is not to build big autonomous AI systems. Instead, the focus is to create simple and efficient NLP agents for small and medium software teams. This changes the goal of agentic AI, not just about making powerful models, but about making them useful in real life. The aim is to ensure that AI support is available even in environments with limited resources.

4 Comparative Analysis of Agentic AI Systems in Software Engineering

This section presents a comparative analysis of representative agentic AI systems, focusing on their applicability to software engineering tasks and their limitations in practical deployment scenarios.

The comparison in Table 3 shows that most NLP based agentic AI systems rely on multi agent architectures and assume stable internet and high computing power, limiting their practicality in resource constrained environments such as freelance work and SMEs. This highlights a gap between current designs and real world usability, emphasizing the need for resource aware NLP agentic systems for software engineering ecosystems.

5 Challenges in Deploying Agentic AI

Despite the rapid advancement of NLP based agentic AI systems, several critical challenges remain that limit their adoption in real world software engineering environments. These challenges shown in Figure 4 become even more noticeable in places like Bangladesh, where limited internet access, lower computing power and different development practices make it harder to use these systems effectively.

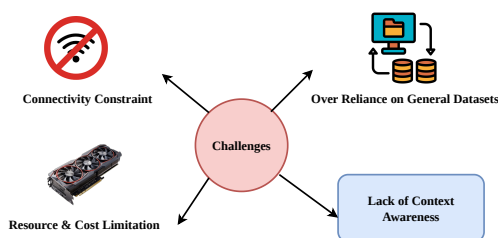


Figure 4: Challenges during Agentic AI Adoption

5.1 Connectivity Constraint

One of the primary challenges is the reliance on continuous internet connectivity. Most Agentic NLP systems are designed for always online environments, making them less effective in regions with unstable or intermittent network access. This creates a gap between system design and actual usage conditions.

5.2 Resource and Cost Limitation

Another major limitation is the high computational and financial cost associated with large scale NLP models. SMEs and freelance developers often lack the infrastructure required to deploy and maintain such systems, restricting their accessibility. Designing efficient NLP agents that operate with reduced computational overhead therefore becomes an important research direction.

5.3 Lack of Context Awareness

Existing systems often struggle to adapt to dynamic and unstructured software development environments. They typically operate well in predefined workflows but lack the flexibility to handle real time changes, informal communication and evolving requirements.

5.4 Over Reliance on General Datasets

Most agentic systems are trained on large, general purpose datasets, which may not reflect the specific needs of local software engineering contexts. This limits their effectiveness in handling region specific tasks and domain specific challenges. Developing localized datasets and domain specific corpora for software engineering communication can significantly improve the effectiveness of NLP driven agentic systems.

6 Future Research Directions and Conclusion

Future research should focus on building lightweight and efficient NLP based agentic systems that can work reliably even under real world constraints such as unstable connectivity and limited computing resources. Exploring hybrid architectures, offline capabilities and task specific optimization techniques can make these systems more practical and accessible. In addition, developing local datasets and designing solutions tailored to freelance developers and SME based software environments can greatly increase the real

Table 3: Comparison of NLP Based Agentic AI Systems for Software Engineering

System	Architecture	Dataset Used	Application	Limitations
GraphAgent (?)	Multi Agent	Not specified / Knowledge Graph	Code + structured reasoning	Limited scalability
AI Software Eng. Agents (?)	Multi Agent	GitHub repositories	Code generation, testing	High computational cost
Automated NLP Pipeline (?)	Single Agent	NLP datasets	Information extraction	Limited autonomy
Autonomous Workflow Agents (?)	Multi Agent	Workflow datasets	Task automation	Requires stable infrastructure
DouAgent (?)	Multi Agent	Simulation dataset	Task decomposition	Coordination complexity
Knowledge Retrieval Agents (?)	Single Agent	Knowledge base datasets	Documentation, retrieval	Limited reasoning depth
Data Analysis Agents (?)	Multi Agent	Analytical datasets	Data analysis	Resource intensive
RUBRA Framework (?)	Single Agent (RAG based)	Retrieval datasets	Educational NLP + reasoning	Domain specific, not generalizable
Multi Agent System Framework (?)	Multi Agent	Simulation dataset	Distributed decision making	Generic design, lacks SE specialization

world usefulness of agentic AI. Such efforts can help bridge the gap between theoretical research and practical software engineering applications.

This paper presents a focused survey on NLP based agentic AI systems in the context of software engineering, highlighting their capabilities, architectures and real world limitations. Unlike traditional surveys, this work emphasizes practical deployment challenges in resource constrained environments such as Bangladesh. Through a comparative analysis and contextual discussion, the study identifies key gaps in current systems, particularly in terms of connectivity dependence, computational requirements and lack of adaptability. To address these challenges, this paper highlights the importance of lightweight, context aware and connectivity resilient NLP based agentic systems.